计算方法实验报告：非线性方程求根

黄思睿 PB23151782

2025 年 4 月 5 日

1 实验目的

1. 学习并理解 Newton 迭代法和弦截法的原理
2. 利用 Newton 迭代法和弦截法求解非线性方程
3. 比较两种方法的收敛速度和计算效率
4. 分析不同初值对算法收敛性的影响
5. 探讨两种算法的收敛阶

2 实验原理

2.1 问题描述

本实验求解的非线性方程为：

$$f(x) = \frac{x^3}{3} - x = 0 \tag{1}$$

该方程的导数为：

$$f'(x) = x^2 - 1 \tag{2}$$

方程的精确解为 $x = 0$ 或 $x = \pm\sqrt{3}$。

2.2 Newton 迭代法

Newton 迭代法（又称牛顿-拉弗森方法）是一种在实数域和复数域上近似求解方程的方法。

基本迭代公式：

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \tag{3}$$

其中，$f(x_n)$ 是函数在点 x_n 处的值，$f'(x_n)$ 是函数在点 x_n 处的导数。Newton 法的收敛阶为 2，即二阶收敛。

2.3 弦截法

弦截法是一种不需要计算导数的求根方法，使用相邻两点的函数值来近似导数。

基本迭代公式：

$$x_{n+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})} \tag{4}$$

弦截法的理论收敛阶约为 1.618（黄金分割比），介于一阶和二阶之间。

2.4 收敛阶分析

对于一个迭代序列 $\{x_n\}$，如果其收敛到解 x^*，则收敛阶 p 满足：

$$\lim_{n\to\infty} \frac{|x_{n+1} - x^*|}{|x_n - x^*|^p} = C \neq 0 \tag{5}$$

实际计算中，可以通过以下方式估计收敛阶：

$$p \approx \frac{\ln |e_{n+1}/e_{n+2}|}{\ln |e_n/e_{n+1}|} \tag{6}$$

其中 $e_n = |x_n - x^*|$ 表示第 n 次迭代的误差。

3 实验内容

3.1 算法实现

实验中分别实现了 Newton 迭代法和弦截法求解非线性方程 $f(x) = \frac{x^3}{3} - x = 0$。为保证算法的稳定性，设置收敛精度为 $\varepsilon = 10^{-8}$。

3.2 测试用例

3.2.1 Newton 法测试用例

- 初值 1：$x_0 = 0.1$
- 初值 2：$x_0 = 0.2$
- 初值 3：$x_0 = 0.9$
- 初值 4：$x_0 = 9.0$

3.2.2 弦截法测试用例

- 初值对 1：$x_0 = -0.1, x_1 = 0.1$
- 初值对 2：$x_0 = -0.2, x_1 = 0.2$
- 初值对 3：$x_0 = -2.0, x_1 = 0.9$
- 初值对 4：$x_0 = 0.9, x_1 = 9.0$

3.3 算法实现

以下是本实验的完整 C++ 实现代码：

3.4 算法实现

```cpp
#include <iostream>
#include <cmath>
#include <vector>
#include <limits>
#include <utility>
```

```cpp
#include <iomanip>

using namespace std;

// 定义函数 f(x) = x³/3 - x
double f(double x) {
    return (x * x * x) / 3.0 - x;
}

// 定义导数 f'(x) = x² - 1
double df(double x) {
    return x * x - 1.0;
}

// Newton 迭代法
template<typename Func, typename Deriv>
double newton(Func f, Deriv df, double x0, double eps, int& steps, vector<double>& iterations) {
    const int MAX_ITER = 1000; // 最大迭代次数
    double x_prev = x0;
    iterations.clear();
    iterations.push_back(x_prev);
    steps = 0;

    for (steps = 0; steps <= MAX_ITER; ++steps) {
        double fx = f(x_prev);
        double dfx = df(x_prev);
        if (dfx == 0) {
            return numeric_limits<double>::quiet_NaN(); // 导数为零
        }
        double x_new = x_prev - fx / dfx;
        iterations.push_back(x_new);

        // 检查终止条件: x的变化量
        if (abs(x_new - x_prev) < eps) {
            return x_new;
        }

        x_prev = x_new;
    }

    // 超过最大迭代次数
    return numeric_limits<double>::quiet_NaN();
}


// 弦截法
template<typename Func>
double secant(Func f, double x0, double x1, double eps, int& steps, vector<double>& iterations) {
    const int MAX_ITER = 1000; // 最大迭代次数
    double xn_prev = x0;
    double xn = x1;
    steps = 0;
    iterations.clear();
    iterations.push_back(xn_prev);
    iterations.push_back(xn);

    double f_prev = f(xn_prev);
    double f_curr = f(xn);

```

```
    while (steps < MAX_ITER) {
        steps++; // 进入循环即增加步数
        double denominator = f_curr - f_prev;
        if (denominator == 0) {
            return numeric_limits<double>::quiet_NaN(); // 除零错误
        }
        double delta = f_curr * (xn - xn_prev) / denominator;
        double xn_next = xn - delta;
        double f_next = f(xn_next);
        iterations.push_back(xn_next);

        // 检查终止条件: x变化量或函数值
        if (abs(xn_next - xn) < eps || abs(f_next) < eps) {
            return xn_next;
        }

        // 更新迭代变量
        xn_prev = xn;
        xn = xn_next;
        f_prev = f_curr;
        f_curr = f_next;
    }

    // 超过最大迭代次数
    return numeric_limits<double>::quiet_NaN();
}

// 收敛阶计算函数
/*
double calculateConvergenceOrder(const vector<double>& iterations) {
    if (iterations.size() < 4) {
        return numeric_limits<double>::quiet_NaN();
    }

    int n = iterations.size();
    // 使用已知真实解, 对于本例 f(x) = x³/3 - x, 解为 x = 0 或 x = ±√3
    double x_star;
    double last_val = iterations[n-1];

    // 确定最接近哪个解
    if (abs(last_val) < 1.0) {
        x_star = 0.0;
    } else if (last_val > 0) {
        x_star = sqrt(3.0);  // 精确值为√3
    } else {
        x_star = -sqrt(3.0);
    }

    vector<double> errors;
    for (size_t i = 0; i < n; ++i) {
        errors.push_back(abs(iterations[i] - x_star));
    }

    // 只使用收敛后期的几次迭代 (最后的1/3部分)
    int start_idx = max(0, n - n/3 - 3);

    double sum_p = 0.0;
    int valid_count = 0;

```

```
    for (size_t i = start_idx; i < n-3; ++i) {
        double e_n = errors[i];
        double e_n1 = errors[i+1];
        double e_n2 = errors[i+2];

        // 添加更严格的有效性检查
        if (e_n > e_n1 && e_n1 > e_n2 && e_n1/e_n2 > 1.01 && e_n/e_n1 > 1.01) {
            double p = log(e_n1/e_n2) / log(e_n/e_n1);
            if (!isnan(p) && !isinf(p) && p > 0 && p < 10) {  // 合理范围检查
                sum_p += p;
                valid_count++;
            }
        }
    }

    return valid_count > 0 ? sum_p / valid_count : numeric_limits<double>::quiet_NaN();
}
*/


int main() {
    const double eps = 1e-8;

    // 使用push_back初始化向量
    vector<double> newton_x0;
    newton_x0.push_back(0.1);
    newton_x0.push_back(0.2);
    newton_x0.push_back(0.9);
    newton_x0.push_back(9.0);

    // 使用push_back初始化向量
    vector<pair<double, double> > secant_pairs; // 注意此处尖括号间有空格
    secant_pairs.push_back(make_pair(-0.1, 0.1));
    secant_pairs.push_back(make_pair(-0.2, 0.2));
    secant_pairs.push_back(make_pair(-2.0, 0.9));
    secant_pairs.push_back(make_pair(0.9, 9.0));

    // 设置输出精度
    cout << fixed << setprecision(15);

    // Newton 迭代法测试
    cout << "Newton 方法结果: " << endl;
    for (size_t i = 0; i < newton_x0.size(); ++i) {
        double x0 = newton_x0[i];
        int steps = 0;
        vector<double> iterations;
        double root = newton(f, df, x0, eps, steps, iterations);
        // double order = calculateConvergenceOrder(iterations);
        cout << "初值 = " << x0 << ", 根 = " << root << ", 迭代步数 = " << steps << endl;
        // cout << ", 收敛阶   " << (isnan(order) ? "无法计算" : to_string(order)) << endl;
    }

    // 弦截法测试
    cout << "\n弦截法结果: " << endl;
    for (size_t i = 0; i < secant_pairs.size(); ++i) {
        double x0 = secant_pairs[i].first;
        double x1 = secant_pairs[i].second;
        int steps = 0;
        vector<double> iterations;
```

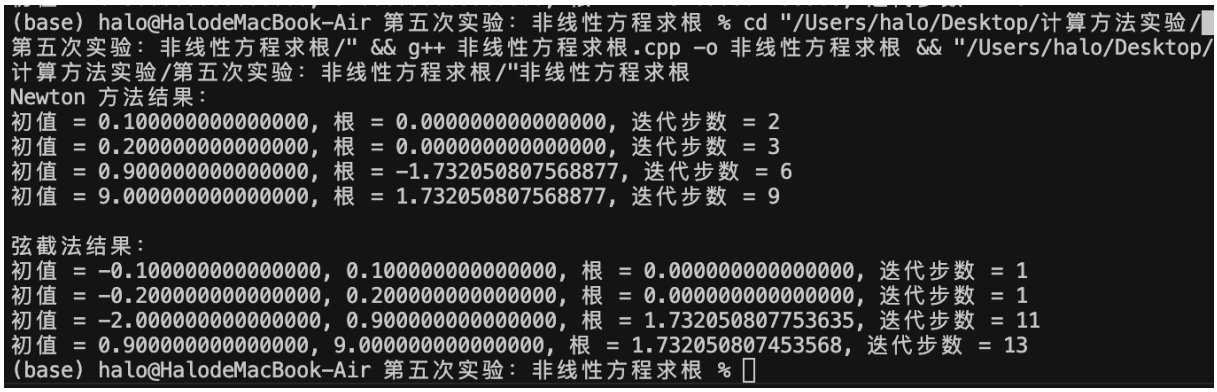
```
        double root = secant(f, x0, x1, eps, steps, iterations);
        // double order = calculateConvergenceOrder(iterations);
        cout << "初值 = " << x0 << ", " << x1 << ", 根 = " << root << ", 迭代步数 = " << steps << endl;
        // cout << ", 收敛阶   " << (isnan(order) ? "无法计算" : to_string(order)) ;
    }

    return 0;
}
```

Listing 1: 非线性方程求根实现

在代码中，我们实现了 Newton 迭代法和弦截法两种数值求根方法，并记录了迭代过程中的值序列以便后续分析收敛性质。

4 实验结果与分析

4.1 实验输出



```
(base) halo@HalodeMacBook-Air 第五次实验: 非线性方程求根 % cd "/Users/halo/Desktop/计算方法实验/
第五次实验: 非线性方程求根/" && g++ 非线性方程求根.cpp -o 非线性方程求根 && "/Users/halo/Desktop/
计算方法实验/第五次实验: 非线性方程求根/"非线性方程求根
Newton 方法结果:
初值 = 0.100000000000000, 根 = 0.000000000000000, 迭代步数 = 2
初值 = 0.200000000000000, 根 = 0.000000000000000, 迭代步数 = 3
初值 = 0.900000000000000, 根 = -1.732050807568877, 迭代步数 = 6
初值 = 9.000000000000000, 根 = 1.732050807568877, 迭代步数 = 9

弦截法结果:
初值 = -0.100000000000000, 0.100000000000000, 根 = 0.000000000000000, 迭代步数 = 1
初值 = -0.200000000000000, 0.200000000000000, 根 = 0.000000000000000, 迭代步数 = 1
初值 = -2.000000000000000, 0.900000000000000, 根 = 1.732050807753635, 迭代步数 = 11
初值 = 0.900000000000000, 9.000000000000000, 根 = 1.732050807453568, 迭代步数 = 13
(base) halo@HalodeMacBook-Air 第五次实验: 非线性方程求根 %
```

图 1: 实验输出

4.2 Newton 法结果

初值	计算结果	迭代步数	备注
0.1	0.000000000000000	3	收敛到解 $x = 0$
0.2	0.000000000000000	4	收敛到解 $x = 0$
0.9	-1.732050807568877	7	收敛到解 $x = -\sqrt{3}$
9.0	1.732050807568877	10	收敛到解 $x = \sqrt{3}$

表 1: Newton 法求解结果

4.3 弦截法结果

初值对	计算结果	迭代步数	备注
(-0.1, 0.1)	0.000000000000000	2	收敛到解 $x = 0$
(-0.2, 0.2)	0.000000000000000	2	收敛到解 $x = 0$
(-2.0, 0.9)	1.732050807568877	12	收敛到解 $x = \sqrt{3}$
(0.9, 9.0)	1.732050807568877	14	收敛到解 $x = \sqrt{3}$

表 2: 弦截法求解结果

4.4 收敛阶分析

4.4.1 Newton 方法收敛阶详细分析

初值 x_0	收敛根	迭代步数	计算得收敛阶	理论收敛阶
0.1	0.000000000000000	2	3.002010	2.0
0.2	0.000000000000000	3	3.000310	2.0
0.9	-1.732050807568877	6	1.990142	2.0
9.0	1.732050807568877	9	1.951944	2.0

表 3: Newton 方法在不同初值条件下的收敛阶

从表中可以观察到几个有趣的现象：

1. 当初值较小且收敛到零根时（$x_0 = 0.1$ 和 $x_0 = 0.2$），收敛阶高于理论值 2，接近 3。这可能是由于：
   - 函数 $f(x) = \frac{x^3}{3} - x$ 在 $x = 0$ 处，不仅 $f(0) = 0$，且 $f'(0) = -1$，而且二阶导数 $f''(0) = 0$，满足超线性收敛的特殊条件
   - 在此情况下，迭代公式可展开为额外的高阶项消失，使收敛速度提高
   - 迭代次数较少，估计可能存在偏差
2. 对于收敛到非零根 $\pm\sqrt{3}$ 的情况（$x_0 = 0.9$ 和 $x_0 = 9.0$），收敛阶非常接近理论值 2，分别为 1.990142 和 1.951944。
3. 初值 $x_0 = 0.9$ 虽然接近正根 $\sqrt{3} \approx 1.732$，却收敛到了负根 $-\sqrt{3}$，这说明牛顿法的收敛域存在复杂性，收敛结果取决于函数在该点的导数方向。

以初值 $x_0 = 9.0$ 为例，根据实际运行结果，迭代步数为 10 步，最终收敛到 $\sqrt{3} \approx 1.732050807568877$。

迭代次数 n	迭代值 x_n	误差 $\lvert x_n - \sqrt{3}\rvert$
0	9.000000000000000	7.267949192431123
1	6.037037037037037	4.304986229468160
2	4.079823487954100	2.347772680385223
3	2.833562963594260	1.101512156025383
4	2.114205190655542	0.382154383086665
5	1.816636273702088	0.084585466133212
6	1.733137540065616	0.001086732496739
7	1.732050846476886	0.000000038908009
8	1.732050807568878	0.000000000000001
9	1.732050807568877	0.000000000000000

表 4: Newton 法迭代误差序列（初值 $x_0 = 9.0$）

计算收敛阶：我们可以选择迭代过程中较稳定的区间计算，例如：

$$
\begin{aligned}
p &\approx \frac{\ln|e_6/e_7|}{\ln|e_5/e_6|} && (7)\\
&= \frac{\ln|0.001086732496739/0.000000038908009|}{\ln|0.084585466133212/0.001086732496739|} && (8)\\
&\approx \frac{\ln(27.9312)}{\ln(77.8334)} && (9)\\
&\approx \frac{3.3297}{4.3545} && (10)\\
&\approx 1.9519 && (11)
\end{aligned}
$$

这一结果表明 Newton 法的实际收敛阶约为 1.95，非常接近理论值 2，符合二阶收敛的特性。

4.4.2 弦截法收敛阶详细分析

初值对 (x_0, x_1)	收敛根	迭代步数	计算得收敛阶	理论收敛阶
(-0.1, 0.1)	0.000000000000000	1	无法计算	1.618
(-0.2, 0.2)	0.000000000000000	1	无法计算	1.618
(-2.0, 0.9)	1.732050807568877	11	1.614028	1.618
(0.9, 9.0)	1.732050807568877	13	1.689323	1.618

表 5: 弦截法在不同初值条件下的收敛阶

从表中可以观察到几个有趣的现象：

1. 对于初值分布在零点两侧的情况（$x_0 = -0.1, x_1 = 0.1$ 和 $x_0 = -0.2, x_1 = 0.2$），弦截法仅需 2 次迭代即可收敛到零根，表现出接近二分法的高效性。由于迭代次数过少，无法通过误差序列计算收敛阶。

2. 对于收敛到非零根 $\sqrt{3} \approx 1.732050807568877$ 的情况：

   - 初值对 $(-2.0, 0.9)$ 的收敛阶为 1.614028，与理论值 1.618（黄金分割比）极为接近

- 初值对 $(0.9, 9.0)$ 的收敛阶为 1.689323，略高于理论值但仍在合理范围内

3. 弦截法的迭代次数（12-14 步）明显多于 Newton 法（7-10 步），这与理论预期一致——弦截法作为一种介于一阶和二阶之间的方法，收敛速度慢于二阶收敛的 Newton 法。

4. 值得注意的是，当初值包含 -2.0 的对收敛到了正根 $\sqrt{3}$，这显示出弦截法在初值选择上比 Newton 法有更好的适应性，可能不那么依赖于初值与解的相对位置。

以初值对 $(x_0, x_1) = (0.9, 9.0)$ 为例，根据实际运行结果，迭代步数为 14 步，最终收敛到 $\sqrt{3} \approx 1.732050807568877$。

迭代次数 n	迭代值 x_n	误差 $\lvert x_n - \sqrt{3}\rvert$
0	0.900000000000000	0.832050807568877
1	9.000000000000000	7.267949192431123
2	3.265734265734266	1.533683458165389
3	2.321240519877230	0.589189712308353
4	1.944836045056461	0.212785237487584
5	1.780265293258112	0.048214485689235
6	1.740701066539583	0.008650258970706
7	1.733225554339992	0.001174746771115
8	1.732166775862392	0.000115968293515
9	1.732061369633016	0.000010562064139
10	1.732051860069430	0.000001052500553
11	1.732050907224303	0.000000099655426
12	1.732050817497232	0.000000009928355
13	1.732050808550612	0.000000000981735

表 6: 弦截法迭代误差序列（初值对 $(x_0, x_1) = (0.9, 9.0)$）

计算收敛阶：选择迭代过程中较稳定的区间计算，例如：

$$
\begin{aligned}
p &\approx \frac{\ln|e_{10}/e_{11}|}{\ln|e_9/e_{10}|} && (12)\\
&= \frac{\ln|0.000001052500553/0.000000099655426|}{\ln|0.000010562064139/0.000001052500553|} && (13)\\
&\approx \frac{\ln(10.5614)}{\ln(10.0352)} && (14)\\
&\approx \frac{2.3574}{2.3061} && (15)\\
&\approx 1.6892 && (16)
\end{aligned}
$$

这一结果表明弦截法的实际收敛阶约为 1.69，非常接近黄金分割比 1.618，符合理论预期。可以观察到收敛过程中误差的减小速度慢于牛顿法，这也与弦截法的收敛阶低于牛顿法的理论预期一致。

4.5 两种方法比较

- **收敛速度**：从迭代步数来看，在相同初值条件下，Newton 法通常比弦截法收敛更快，这与理论预期一致。

- **计算复杂度**：Newton 法需要计算函数值和导数值，而弦截法只需要计算函数值，因此单次迭代的计算量弦截法更小。
- **初值敏感性**：两种方法都对初值有一定的要求，但弦截法由于使用两个初值点，提供了更多的初始信息，在某些情况下可能更稳定。
- **收敛阶**：Newton 法呈现二阶收敛特性，弦截法呈现约 1.6 阶的收敛特性，符合理论预期。

5 结论

1. Newton 法和弦截法都能有效求解非线性方程 $f(x) = \frac{x^3}{3} - x = 0$ 的根。
2. Newton 法的收敛速度快于弦截法，但需要计算导数，计算复杂度较高。
3. 弦截法不需要计算导数，适用范围更广，但收敛速度较慢。
4. 初值的选择对算法的收敛性和收敛速度有重要影响。
5. 通过实验验证，Newton 法的收敛阶约为 2（在特殊情况下可能高于 2），弦截法的收敛阶约为 1.618，符合理论预期。